

EXPERIMENT 2

Title: React Hooks and Reusable Custom Hooks

AIM

To use **React Hooks and reusable Custom Hooks** to manage forms, car inventory, filters, favourites, comparison and recently viewed cars in the **Carventory** application.

THEORY

React Hooks are functions provided by React that allow functional components to use features such as state and lifecycle behaviour. Hooks make it easier to manage application logic without using class components.

The most commonly used Hook is **useState**, which is used to store and update data inside a component. Another important Hook is **useEffect**, which is used for performing side effects such as fetching data, updating storage or responding to changes in state.

In larger applications, repeating the same logic in multiple components can make the code difficult to maintain. To solve this problem, React allows developers to create **Custom Hooks**. A Custom Hook is a reusable JavaScript function whose name normally starts with use. It can contain React Hooks and application-specific logic.

In the Carventory application, several Custom Hooks were created:

1. **useCars** – Manages car inventory data and car-related operations.
2. **useCarFilters** – Handles searching, filtering and sorting of cars.
3. **useFavourites** – Manages the cars saved by the customer as favourites.
4. **useCompare** – Handles selecting and comparing multiple cars.
5. **useRecentlyViewed** – Keeps track of cars recently viewed by the customer.
6. **useLocalStorage** – Provides reusable logic for storing and retrieving application data from browser local storage.

These Custom Hooks separate the **application logic from the user interface**. This makes the code more reusable, organized and easier to maintain.

The Admin Inventory section was also developed during this experiment. It allows the administrator to perform operations such as:

- Add a car
- Edit car information
- Delete a car
- Change car status
- Validate car information

At this stage, the application uses local/mock data and local storage for demonstration purposes. A real REST API and PostgreSQL database are introduced in later experiments.

IMPLEMENTATION / WORKING

The Carventory frontend was enhanced using React Hooks and Custom Hooks.

The major functionality implemented includes:

- Managing car inventory using useCars
- Searching and filtering cars using useCarFilters
- Saving favourite cars using useFavourites
- Selecting cars for comparison using useCompare
- Tracking recently viewed cars using useRecentlyViewed
- Persisting selected data using useLocalStorage
- Managing Admin Inventory operations
- Form validation for adding and editing cars

For example, when a customer marks a car as a favourite, the favourite state is updated through the useFavourites Hook. Similarly, when a customer opens a car, it can be recorded using the useRecentlyViewed Hook.

This approach avoids duplicating the same logic in different pages and components.

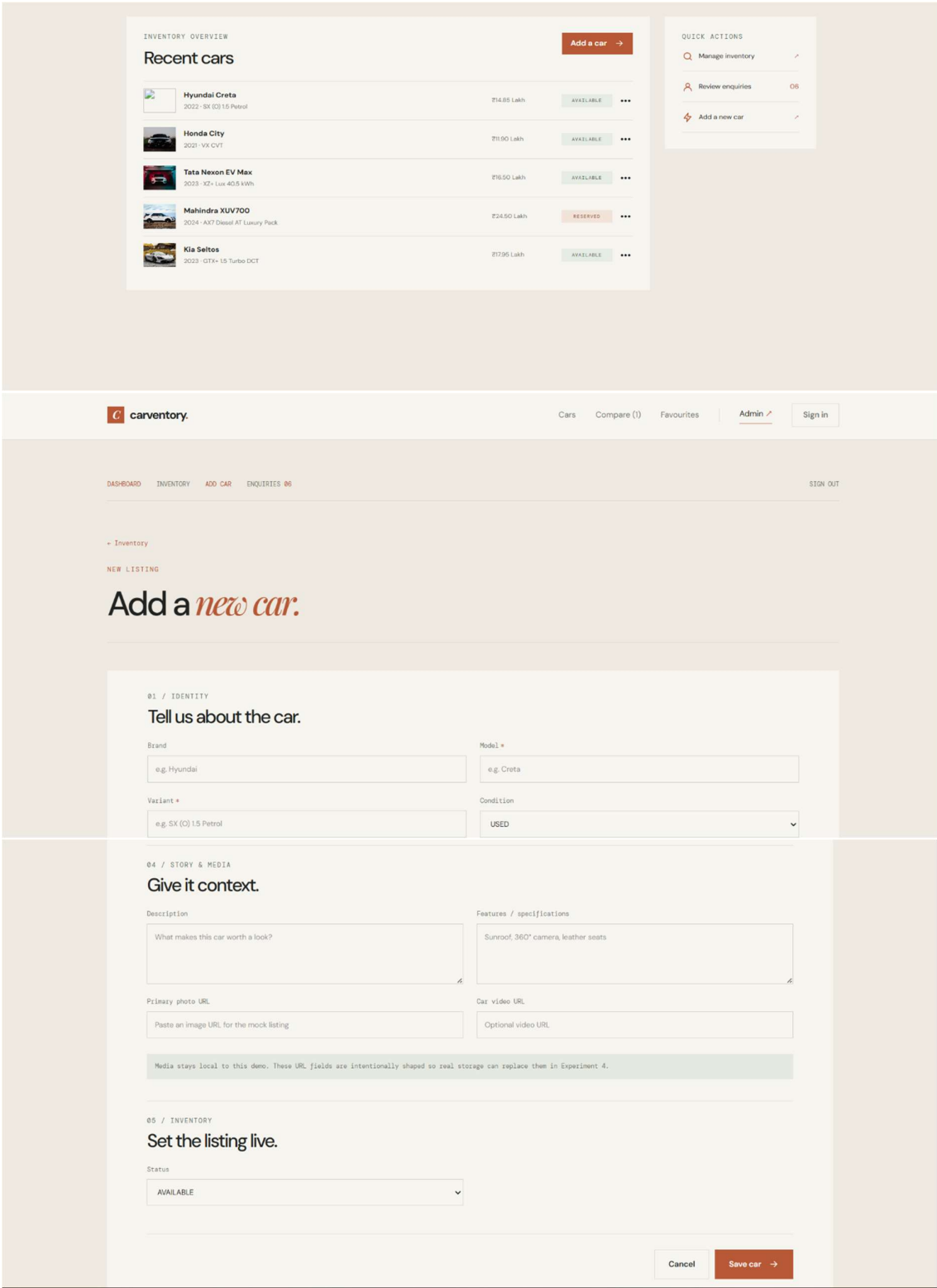
OUTPUT

The following functionality was successfully implemented in Carventory:

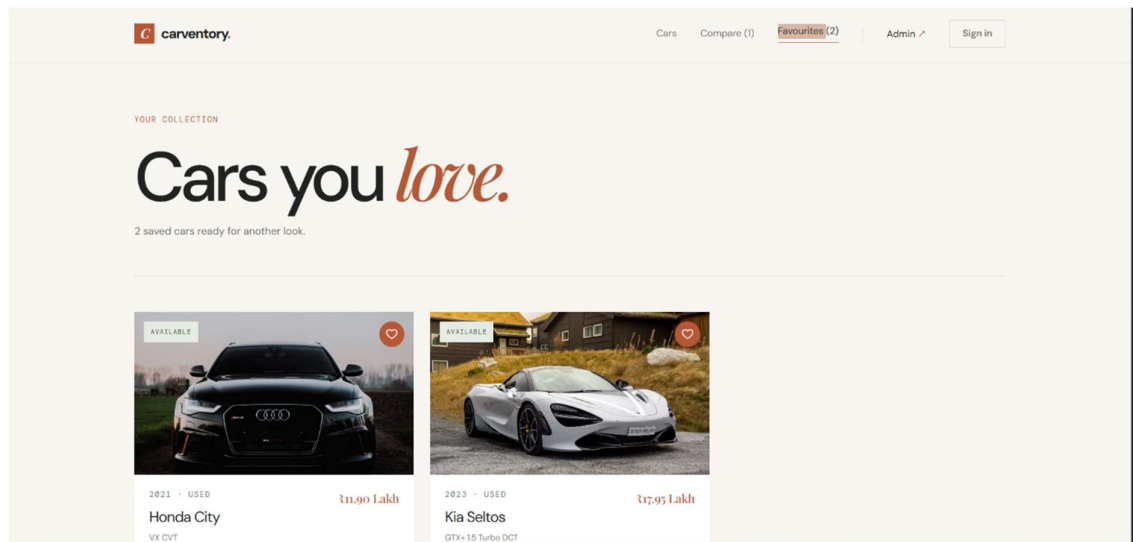
- Favourite cars
- Compare cars
- Recently viewed cars
- Local storage persistence
- Admin Login interface
- Add Car form
- Edit Car functionality
- Delete Car functionality
- Form validation
- Car status management
- Responsive interface

Output Screenshot

ADD FEATURE:



FAVOURITE FEATURE:



CONCLUSION

Thus, **React Hooks and reusable Custom Hooks** were successfully implemented in the Carventory application. Hooks were used to manage inventory, filtering, favourites, comparison, recently viewed cars and local storage. Admin inventory operations and form validation were also implemented.

The use of Custom Hooks made the application logic **reusable, organized and easier to maintain**, providing a strong foundation for global state management in the next experiment.